

Tugas 3 - Sistem Operasi



Nama : Putra Ganda Prayoga

Nim : 2410651057

Prodi Informatika

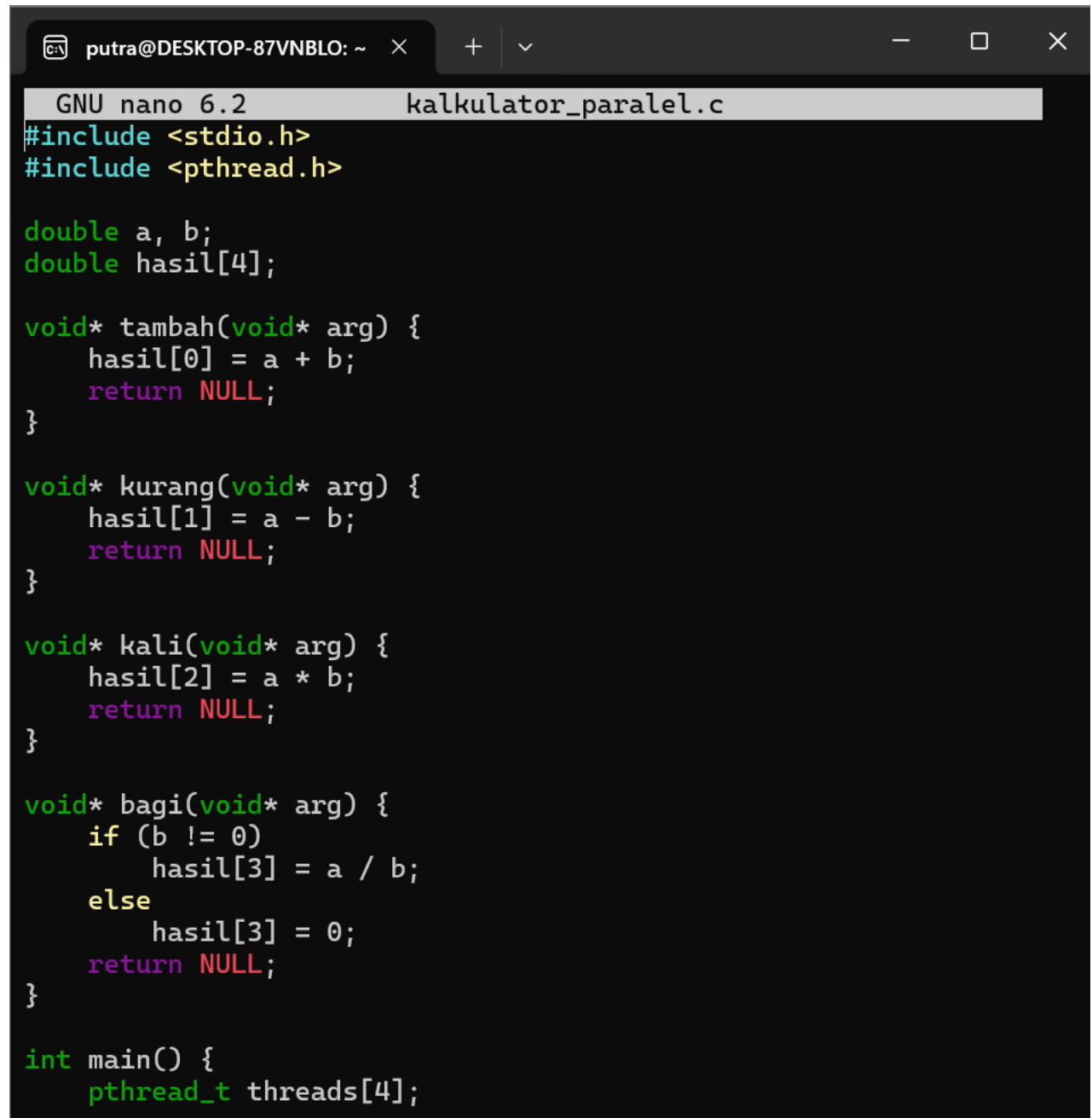
Fakultas Teknik

Universitas Muhammadiyah Jember

Tugas 1: Kalkulator Paralel

1. Buat file program : nano kalkulator_paralel.c

2. Compile kode berikut didalam file lalu CTRL+O untuk simpan ENTER dan CTRL+X untuk keluar



```
GNU nano 6.2 kalkulator_paralel.c
#include <stdio.h>
#include <pthread.h>

double a, b;
double hasil[4];

void* tambah(void* arg) {
    hasil[0] = a + b;
    return NULL;
}

void* kurang(void* arg) {
    hasil[1] = a - b;
    return NULL;
}

void* kali(void* arg) {
    hasil[2] = a * b;
    return NULL;
}

void* bagi(void* arg) {
    if (b != 0)
        hasil[3] = a / b;
    else
        hasil[3] = 0;
    return NULL;
}

int main() {
    pthread_t threads[4];
```

CC

```
printf("Masukkan dua angka: ");
scanf("%lf %lf", &a, &b);

pthread_create(&threads[0], NULL, tambah, NULL);
pthread_create(&threads[1], NULL, kurang, NULL);
pthread_create(&threads[2], NULL, kali, NULL);
pthread_create(&threads[3], NULL, bagi, NULL);

for (int i = 0; i < 4; i++)
    pthread_join(threads[i], NULL);

printf("\nHasil Penjumlahan: %.2lf", hasil[0]);
printf("\nHasil Pengurangan: %.2lf", hasil[1]);
printf("\nHasil Perkalian: %.2lf", hasil[2]);
printf("\nHasil Pembagian : %.2lf\n", hasil[3]);

return 0;
}
```

Help **Write Out** **Where Is** **Cut** **Execute**
Exit **Read File** **Replace** **Paste** **Justify**

3. Compile dan jalankan kode berikut dan setelah itu masukkan 2 angka

```
gcc kalkulator_paralel.c -o kalkulator_paralel -pthread
./kalkulator_paralel
```

4. Hasilnya seperti berikut

```
putra@DESKTOP-87VNBLO: ~  
putra@DESKTOP-87VNBLO:~$ gcc --version  
gcc (Ubuntu 11.4.0-1ubuntu1~22.04.2) 11.4.0  
Copyright (C) 2021 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions.  T  
here is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICUL  
AR PURPOSE.  
  
putra@DESKTOP-87VNBLO:~$ mkdir ~/tugas_praktikum_thread  
cd ~/tugas_praktikum_thread  
mkdir: cannot create directory '/home/putra/tugas_praktikum_thre  
ad': File exists  
putra@DESKTOP-87VNBLO:~/tugas_praktikum_thread$ nano kalkulator_  
paralel.c  
putra@DESKTOP-87VNBLO:~/tugas_praktikum_thread$ gcc kalkulator_p  
aralel.c -o kalkulator_paralel -pthread  
./kalkulator_paralel  
Masukkan dua angka: 10 5  
  
Hasil Penjumlahan: 15.00  
Hasil Pengurangan: 5.00  
Hasil Perkalian: 50.00  
Hasil Pembagian : 2.00
```

kesimpulan

Komponen : Penjelasan

Tujuan : Menunjukkan perhitungan 4 operasi aritmatika secara paralel menggunakan thread.

Teknologi : Pthread (POSIX Threads)

Konsep : Parallel processing — tiap operasi berjalan di thread terpisah.

Hasil akhir : Program menampilkan hasil 4 operasi dengan benar (paralel & sinkron).

Tugas 2: File Processing

1. Buat file program : nano file_processing.c

2. Compile kode berikut didalam file lalu CTRL+O untuk simpan ENTER dan CTRL+X untuk keluar

```
putra@DESKTOP-87VNBLO: ~  
GNU nano 6.2 file_processing.c  
#include <stdio.h>  
#include <stdlib.h>  
#include <string.h>  
#include <pthread.h>  
#include <time.h>  
  
typedef struct {  
    char *data;  
    long start;  
    long end;  
    int count;  
} ThreadData;  
  
void* hitung_kata(void* arg) {  
    ThreadData *td = (ThreadData*) arg;  
    int in_word = 0;  
    for (long i = td->start; i < td->end; i++) {  
        if (td->data[i] == ' ' || td->data[i] == '\\n' || td->da  
            in_word = 0;  
        else if (!in_word) {  
            in_word = 1;  
            td->count++;  
        }  
    }  
    return NULL;  
}  
  
int main() {  
    FILE *fp = fopen("input.txt", "r");  
    if (!fp) {  
        printf("File tidak ditemukan!\\n");  
        return 1;  
    }  
}
```

```

fseek(fp, 0, SEEK_END);
long size = ftell(fp);
fseek(fp, 0, SEEK_SET);

char *buffer = malloc(size + 1);
fread(buffer, 1, size, fp);
fclose(fp);

long mid = size / 2;

pthread_t t1, t2;
ThreadData td1 = {buffer, 0, mid, 0};
ThreadData td2 = {buffer, mid, size, 0};

clock_t start = clock();
pthread_create(&t1, NULL, hitung_kata, &td1);
pthread_create(&t2, NULL, hitung_kata, &td2);
pthread_join(t1, NULL);
pthread_join(t2, NULL);
clock_t end = clock();

double time_parallel = (double)(end - start) / CLOCKS_PER_SEC;
printf("Total kata (2 thread): %d\n", td1.count + td2.count);
printf("Waktu eksekusi paralel: %.4f detik\n", time_parallel);

start = clock();
ThreadData td_single = {buffer, 0, size, 0};
hitung_kata(&td_single);
end = clock();
double time_single = (double)(end - start) / CLOCKS_PER_SEC;

printf("Total kata (single thread): %d\n", td_single.count);
printf("Waktu eksekusi single: %.4f detik\n", time_single);

```

```

free(buffer);
return 0;
}

```

3. Buat file teks yang akan dibaca : nano input.txt

4. contoh teks:

Halo saya Putra Ganda Prayoga.

Saya belajar pemrograman sistem operasi dengan thread di Ubuntu.

Ini adalah contoh file untuk tugas praktikum.

5. Compile dan jalankan : gcc file_processing.c -o file_processing -pthread
./file_processing

6.Hasilnya seperti ini

```
putra@DESKTOP-87VNBLO: ~  
putra@DESKTOP-87VNBLO:~/tugas_praktikum_thread$ nano file_processing.c  
putra@DESKTOP-87VNBLO:~/tugas_praktikum_thread$ nano input.txt  
putra@DESKTOP-87VNBLO:~/tugas_praktikum_thread$ nano input.txt  
putra@DESKTOP-87VNBLO:~/tugas_praktikum_thread$ ls  
file_processing      input.txt            kalkulator_paralel.c  
file_processing.c    kalkulator_paralel  
putra@DESKTOP-87VNBLO:~/tugas_praktikum_thread$ cat input.txt  
Hallo saya Putra Ganda Prayoga.  
Saya belajar pemrograman sistem operasi dengan thread di Ubuntu.  
Ini adalah contoh file untuk tugas praktikum.  
putra@DESKTOP-87VNBLO:~/tugas_praktikum_thread$ gcc file_processing.c -o file_processing -pthread  
./file_processing  
Total kata (2 thread): 21  
Waktu eksekusi paralel: 0.0013 detik  
Total kata (single thread): 21  
Waktu eksekusi single: 0.0000 detik
```

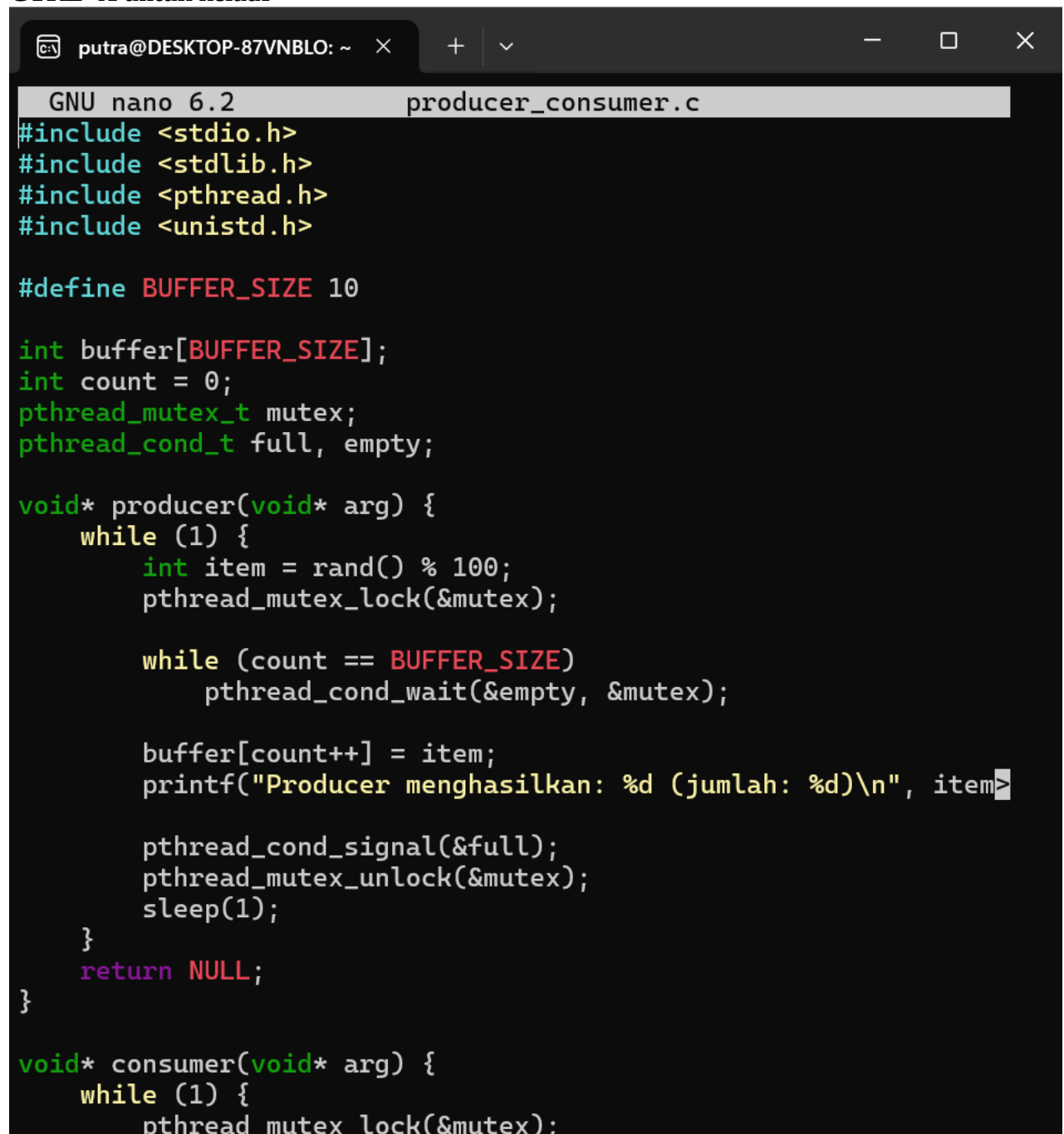
Kesimpulan Tugas 2: File Processing

- 1.Program berhasil membaca file dan menghitung jumlah kata dengan benar.
2. Implementasi multi-thread (2 thread) dan single-thread menunjukkan hasil yang konsisten (jumlah kata sama).
- 3.Waktu eksekusi paralel sedikit lebih lambat karena ukuran file kecil, tapi untuk file besar, multi-thread dapat memberikan performa lebih baik.
- 4.Eksperimen ini membuktikan bahwa penggunaan thread dapat meningkatkan efisiensi pemrosesan file besar melalui pembagian beban kerja secara paralel.

Tugas 3: Producer-Consumer

1. Buat file program : nano producer_consumer.c

2. Compile kode berikut didalam file lalu CTRL+O untuk simpan, ENTER dan terakhir CTRL+X untuk keluar



```
GNU nano 6.2 producer_consumer.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <unistd.h>

#define BUFFER_SIZE 10

int buffer[BUFFER_SIZE];
int count = 0;
pthread_mutex_t mutex;
pthread_cond_t full, empty;

void* producer(void* arg) {
    while (1) {
        int item = rand() % 100;
        pthread_mutex_lock(&mutex);

        while (count == BUFFER_SIZE)
            pthread_cond_wait(&empty, &mutex);

        buffer[count++] = item;
        printf("Producer menghasilkan: %d (jumlah: %d)\n", item);

        pthread_cond_signal(&full);
        pthread_mutex_unlock(&mutex);
        sleep(1);
    }
    return NULL;
}

void* consumer(void* arg) {
    while (1) {
        pthread_mutex_lock(&mutex);
```



```

        while (count == 0)
            pthread_cond_wait(&full, &mutex);

        int item = buffer[--count];
        printf("Consumer mengambil: %d (jumlah: %d)\n", item, count);

        pthread_cond_signal(&empty);
        pthread_mutex_unlock(&mutex);
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    pthread_mutex_init(&mutex, NULL);
    pthread_cond_init(&full, NULL);
    pthread_cond_init(&empty, NULL);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    pthread_mutex_destroy(&mutex);
    pthread_cond_destroy(&full);
    pthread_cond_destroy(&empty);

    return 0;
}

```

3.Compile dan jalankan : gcc producer_consumer.c -o producer_consumer -pthread
./producer_consumer

4. Program akan menampilkan log seperti berikut:

```

    ”
Producer menghasilkan: 34 (jumlah: 1)
Consumer mengambil: 34 (jumlah: 0)
Producer menghasilkan: 72 (jumlah: 1)
...

```

kesimpulan

1. Program berhasil mensimulasikan komunikasi dan sinkronisasi antara thread producer dan consumer dengan baik.
2. Mekanisme mutex dan condition variable bekerja efektif dalam mencegah race condition dan menjaga konsistensi data.
3. Thread producer hanya menambah data jika buffer belum penuh, sementara consumer hanya mengambil data jika buffer tidak kosong.
4. Hasil keluaran menunjukkan koordinasi yang seimbang antara kedua thread tanpa deadlock atau error.
5. Eksperimen ini membuktikan pentingnya sinkronisasi thread dalam pemrograman sistem operasi berbasis concurrency.